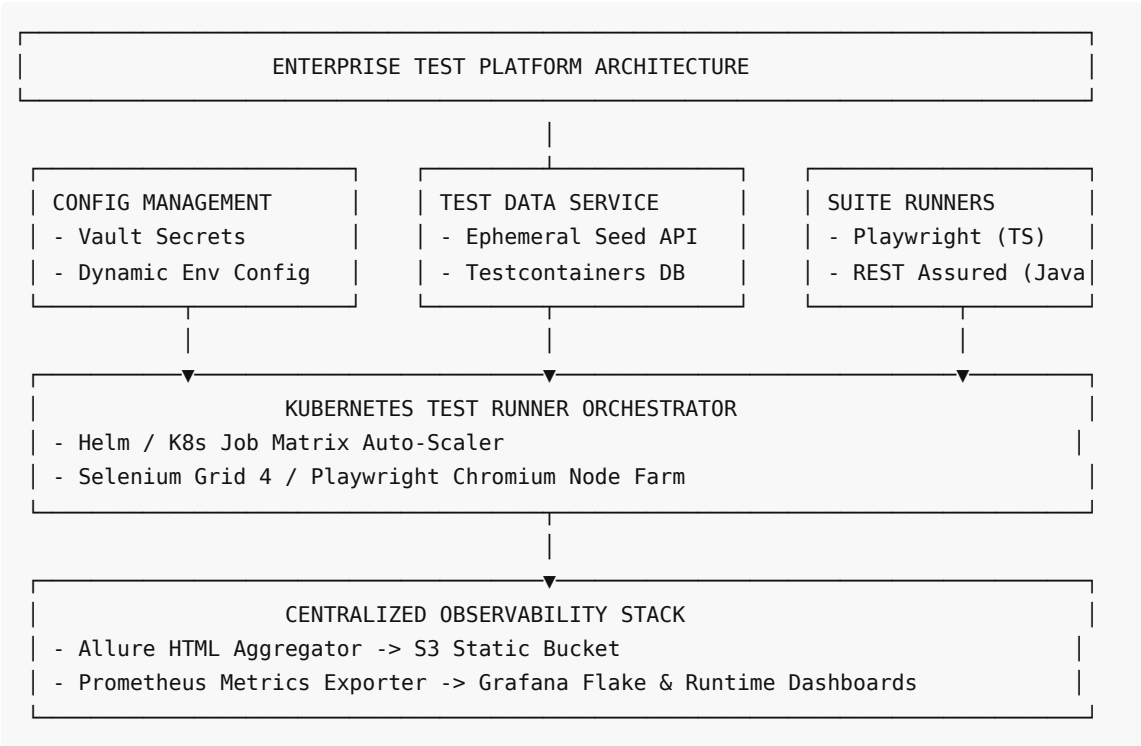


SECTION 32 — SYSTEM DESIGN FOR SDET

Purpose: System architecture, distributed test infrastructure design, multi-tenant execution platforms, and enterprise observability engineering for Enterprise Senior SDET / Staff QA Architect assessment (Deloitte, Big 4, FAANG).

32.1 Enterprise Test Automation Infrastructure Blueprint



32.2 Multi-Tenant Data & Environment Isolation Strategy

Isolation Boundaries Matrix

Isolation Level	Implementation Mechanism	Pros / Cons
Thread Level	ThreadLocal driver & unique UUID per worker	Fast (<1ms); requires strict cleanup
Context Level	Playwright BrowserContext in-memory storage	Isolated cookies/storage (<30ms setup)
Database Level	Docker Testcontainers PostgreSQL per worker	100% data parity; requires Docker daemon
Tenant Level	Dynamic Tenant ID header + API Data Factory	Production-grade isolation; requires API hooks

32.3 Distributed Test Execution Engine (10,000+ Tests/Day)

Architectural Requirements

1. **Dynamic Scaling:** Provision ephemeral Kubernetes pods for test execution on demand; terminate pods upon completion to optimize cloud compute costs.
2. **Deterministic Sharding:** Partition test specs across N parallel runners based on past execution duration history (LPT: Longest Processing Time First algorithm).
3. **Artifact Streaming:** Stream video recordings, traces, and console logs asynchronously to AWS S3 bucket during execution without blocking main runner threads.

```
// Architectural Implementation: Dynamic LPT Test Sharding Algorithm
interface TestSpec {
  path: string;
  historicalDurationMs: number;
}

function distributeShards(specs: TestSpec[], totalShards: number): TestSpec[][] {
  // 1. Sort specs descending by duration (Longest Processing Time First)
  const sorted = [...specs].sort((a, b) => b.historicalDurationMs - a.historicalDurationMs);

  const shards: TestSpec[][] = Array.from({ length: totalShards }, () => []);
  const shardDurations: number[] = new Array(totalShards).fill(0);

  // 2. Greedy assignment to least loaded shard
  for (const spec of sorted) {
    const minShardIndex = shardDurations.indexOf(Math.min(...shardDurations));
    shards[minShardIndex].push(spec);
    shardDurations[minShardIndex] += spec.historicalDurationMs;
  }

  return shards;
}
```

32.4 System Observability & Telemetry Integration

Telemetry Architecture

- **Metrics Collection:** Prometheus Exporter scraping JUnit XML test results.
- **Dashboards:** Grafana visualizing:
 - *Pass/Fail Trends* by branch and commit.
 - *P95 Test Duration Trends* by spec file.
 - *Flake SLA Compliance* over 30-day windows.
- **Alerting:** Automated Slack / PagerDuty alerts triggered when suite failure rate exceeds 5% or execution duration increases by $> 20\%$.

```
# Prometheus Metric Scraping Configuration for Test Metrics
scrape_configs:
  - job_name: 'test_automation_metrics'
    static_configs:
      - targets: ['test-exporter.staging.internal:9090']
```

```
metrics_path: '/metrics'  
scrape_interval: 15s
```

32.5 Senior System Design Whiteboard Questions & Answers

Question: "How do you architect an automation platform for 10,000 daily tests executing across 4 target environments with zero data collisions?"

- **Answer (Staff/Principal Level):**

1. *Infrastructure:* K8s EKS runner pool using dynamic Helm job provisioning.
2. *Sharding:* Duration-aware LPT sharding algorithm distributing specs across 32 worker nodes.
3. *Data Isolation:* ApiDataFactory creating ephemeral accounts using unique UUID strings (user_\${worker_id}_\${timestamp}@test.com).
4. *Configuration:* HashiCorp Vault injecting environment credentials dynamically per job run.
5. *Observability:* Unified Allure reporting merged via S3 + Prometheus/Grafana pipeline monitoring.

32.6 Anti-Patterns & Critical Pitfalls

- **Single Monolithic Runner:** Running 10,000 tests on a single VM instead of sharding across Kubernetes pod matrix.